



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

PROYECTO DE BASE DE DATOS II

SafeBridge: Orquestador Multi-Motor de RespalDOS y Validación de Integridad

Informe de Visión del Producto

FD02

CURSO	Base de Datos II
DOCENTE	Ing. Patrick José Cuadros Quiroga
EMPRESA / EQUIPO	BitCraft Solutions
INTEGRANTE	Sierra Ruiz, Iker Alberto (2023077090)
INTEGRANTE	Cortez Mamani, Julio Samuel (2023077283)

Sistema SafeBridge: Orquestador Multi-Motor de Respaldos y Validación de Integridad

Informe FD02 — Roadmap, Características y Gestión del Producto

VERSIÓN 3.0

Control de versiones

Ver-sión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	06/05/2026	Versión Original
2.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	31/05/2026	Actualización de Características y Roadmap a Tauri MVP
3.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	04/07/2026	Revisión completa con ingeniería inversa del código fuente

Índice General

- 1. Descripción General del Producto
- 2. Características Actuales del Producto
 - 2.1 Conexión Multi-Motor
 - 2.2 Validación Nativa (EOF y SHA-256)
 - 2.3 Cifrado de Credenciales con AES-GCM
 - 2.4 Interfaz Gráfica con React y Tailwind
 - 2.5 Sistema de Logging Detallado en SQLite
 - 2.6 Arquitectura Tauri (Backend Rust + Frontend Web)
- 3. Roadmap y Futuras Versiones
- 4. Gestión de Tareas con GitHub Projects y Ramas
 - 4.1 Organización en GitHub Projects
 - 4.2 Estrategia de Ramas (Branching Strategy)
 - 4.3 Ciclo de Vida de una Tarea
- 5. Conclusiones

1. Descripción General del Producto

SafeBridge es una aplicación de escritorio moderna y ultra-eficiente desarrollada en **Tauri v2 (Rust)** con interfaz gráfica en **React y TailwindCSS**, diseñada para ayudar a desarrolladores individuales y pequeñas empresas a orquestar y automatizar el ciclo de generación de *backups* de bases de datos.

A diferencia de las soluciones de terminal comunes, SafeBridge abstrae el uso de los comandos nativos (`pg_dump`, `mysqldump`, etc.) centralizando las credenciales de múltiples motores de bases de datos. El MVP actual genera el archivo `.bak`, `.sql` o `.archive` deseado, calcula su hash SHA-256 de forma ultrarrápida y verifica nativamente las firmas de finalización de archivo (EOF) para garantizar que el sistema no ha producido un archivo truncado o corrupto, registrando el resultado en un historial local.

El producto se enmarca en la filosofía de **continuidad del negocio (BCDR)** adaptado a un entorno de desarrollo local (Local First).

2. Características Actuales del Producto

*Esta sección sirve como contenido base para las **GitHub Wikis** del repositorio oficial del proyecto.*

2.1 Conexión Multi-Motor

SafeBridge implementa soporte para **cuatro motores de base de datos principales** en la versión actual MVP:

Motor	Sidecar (Herramienta)	Archivo Destino
SQL Server	<code>sqlcmd</code>	<code>.sql</code> / <code>.bak</code>
MySQL	<code>mysqldump</code>	<code>.sql</code>
PostgreSQL	<code>pg_dump</code>	<code>.sql</code>
MongoDB	<code>mongodump</code>	<code>.archive</code>

El proceso invoca a los comandos subyacentes empleando la API `tauri_plugin_shell` de Rust, lo que permite el streaming seguro del estándar de salida y error hacia el frontend para que el usuario observe el progreso en tiempo real.

2.2 Validación Nativa (EOF y SHA-256)

En el MVP, en lugar de recurrir a levantar contenedores completos, se implementó una estrategia de validación nativa (función `verify_backup` en `backup.rs`):

- El sistema evalúa el tamaño final del archivo generado.
- Se leen los últimos 256 bytes del archivo sin cargar el archivo completo en RAM.
- Se busca la **firma de conclusión exitosa** del motor correspondiente (ej: `Dump completed on` para MySQL, o `PostgreSQL database dump complete` para PostgreSQL).
- De forma síncrona, se genera un Hash criptográfico **SHA-256** calculando por chunks (bloques de 8KB) todo el archivo en el disco, garantizando la detección rápida de cualquier alteración futura del backup.

2.3 Cifrado de Credenciales con AES-GCM

Las credenciales de conexión (usuario, contraseña) **nunca se almacenan en texto plano en la base local**. SafeBridge implementa cifrado simétrico avanzado mediante la librería `aes-gcm` en Rust:

- Todas las contraseñas se encriptan al ingresar mediante `crypto::encrypt_password`.
- Se guardan en la base SQLite y se desencriptan al vuelo `crypto::decrypt_password` sólo en el momento exacto en el que Tauri debe inyectar la clave en el comando del motor (`PGPASSWORD`, `MYSQL_PWD`).
- El frontend React jamás recibe la contraseña de retorno; el payload de `list_connections` oculta intencionalmente el campo `password`.

2.4 Interfaz Gráfica con React y Tailwind

La capa de presentación utiliza un entorno Node.js / Vite impulsado por **React y Tailwind CSS**. Presenta un diseño "industrial / técnico":

Pantalla	Descripción
Dashboard	Vista rápida de resumen de todas las operaciones realizadas.
Conexiones	CRUD (Crear, Leer, Actualizar, Borrar) de los servidores de bases de datos.
Backup	Selector de base de datos para disparar backups manuales, con panel terminal para logs en tiempo real.
Historial	Tabla detallada mostrando las fechas, tiempos, rutas, estados (OK/FAIL) y el SHA-256 de todas las copias previas.

2.5 Sistema de Logging Detallado en SQLite

En lugar de simples archivos de texto, la aplicación usa **SQLite** (`rusqlite`) local (`safebridge.db`) en el directorio de la aplicación del usuario. La tabla `backup_logs` actúa como libro mayor e inmutable, almacenando: - Identificador UUID. - Duración de la operación (segundos). - Tamaño en disco, Ruta, Status (OK/FAIL), Mensaje de Error (si falló). - Verificación (True/False) y el log de terminal en crudo (`full_logs`).

2.6 Arquitectura Tauri (Backend Rust + Frontend Web)

El software sigue la convención Tauri, separando de forma tajante: - **El Frontend (UI Layer)** solo envía `invoke("generate_backup", { connectionId })` o escucha eventos asíncronos (`app.listen("backup_log")`). - **El Backend (Rust Layer)** posee el control absoluto de los hilos de sistema, escritura en SQLite y ejecución en el shell local. La memoria compartida se gestiona cuidadosamente usando `std::sync::Mutex` (ej. `Mutex<Connection>`).

3. Roadmap y Futuras Versiones

La siguiente tabla presenta el plan de evolución técnica.

Ver-sión	Release Date (estimado)	Estado	Características Principales
v1.0 (MVP)	Mayo 2026	Librada	Tauri v2, SQLite, React + Tailwind. Soporte de PostgreSQL, MySQL, SQLServer, MongoDB. Cifrado AES-GCM. Validación Nativa de EOF y logueo de SHA-256.

Ver-sión	Release Date (estimado)	Estado	Características Principales
v2.0	Agosto 2026	Planificada	Validación Avanzada con Docker: Reemplazar o aumentar la validación EOF con la capacidad teórica descrita de <code>plan.md</code> : Instanciar automáticamente en Docker Desktop contenedores efímeros del motor correspondiente para restaurar las tablas temporalmente y confirmar datos.
v2.5	Octubre 2026	Planificada	Exportación S3 Automática: Integrar AWS SDK para Rust para automatizar el volcado directamente hacia un bucket remoto al finalizar el backup de forma exitosa. Notificaciones de bandeja de sistema (Windows Toast).

4. Gestión de Tareas con GitHub Projects y Ramas

4.1 Organización en GitHub Projects

El desarrollo de SafeBridge sigue apoyándose en **GitHub Projects** con tablero tipo **Kanban**:

Columna	Descripción
Backlog	Issues registradas pero no priorizadas para el sprint actual
Sprint Actual	Tareas comprometidas para la iteración en curso (2 semanas)
En Progreso	Issues actualmente en desarrollo, asignadas a un desarrollador
En Revisión	Pull Requests abiertos, pendientes de code review
Hecho	Issues cerradas y mergeadas a la rama principal

4.2 Estrategia de Ramas (Branching Strategy)

Se adopta **GitHub Flow** simplificado:

```

main                ← Rama de producción y CI de compilación de binario Tauri.
|
├─ feature/rust-crypto      ← Nuevas funcionalidades de Rust backend
├─ feature/react-dashboard  ← Tareas de Frontend
├─ bugfix/postgres-sidecar  ← Corrección de errores
└─ release/v1.0            ← Tag para desencadenar el pipeline de release

```

Reglas de protección: `main` requiere de aprobación obligatoria. El pipeline automatizado de GitHub Actions compila la app y falla si `cargo check` o `npm run build` encuentran errores.

4.3 Ciclo de Vida de una Tarea

1. Issue creada en el proyecto con criterios de aceptación claros.
2. `git checkout -b feature/nueva-db-mysql main`
3. Desarrollo local (modificaciones en `/src-tauri/src/` o `/src/`).
4. Commit empleando la convención (ej: `feat(rust): add mysqldump integration`).
5. Pull Request a `main`. Code Review cruzado entre desarrolladores (Frontend evalúa la UI, Backend evalúa el código Rust).
6. Squash Merge y cierre automático de la tarea.

5. Conclusiones

El cambio arquitectónico hacia **Tauri y Rust** garantiza un nivel de seguridad y rendimiento sumamente superior frente a otros frameworks de escritorio interpretados. El producto v1.0 (MVP) implementa un ciclo de orquestación de bases de datos maduro, solucionando con éxito el riesgo constante de tener "archivos `.bak` que no funcionan" mediante comprobaciones nativas automáticas en una interfaz gráfica atractiva para desarrolladores individuales.

El roadmap proyecta la incorporación de integración directa con contenedores Docker para un testeo profundo, alineando el desarrollo del proyecto con el estado del arte de la industria en Continuidad de Negocios (BCDR).

Documento actualizado por el equipo BitCraft Solutions — Universidad Privada de Tacna, FAING-EPIS, Ciclo 2026-I.